Syscall catalog (generated)

This table is generated from the syscall numbers in `kernel/include/kernel/syscall.h` and the userspace wrappers in `user/libc/unistd.c`. Calling convention: `int 0x80` with `eax=syscall_no`, args in `ebx/ecx/edx`, return in `eax` (typically `-1` on error).

#	Syscall macro	Userspace wrapper	Register mapping (from wrapper)
1	`SYSCALL_WRITE`	`write(const void *buf, uint32_t len)`	`eax=SYSCALL_WRITE, ebx=(uint32_t)buf, ecx=len, edx=0`
2	`SYSCALL_EXIT`	``	`eax=SYSCALL_EXIT, ebx=?, ecx=?, edx=?`
3	`SYSCALL_OPEN`	`open(const char *path)`	`eax=SYSCALL_OPEN, ebx=(uint32_t)path, ecx=0, edx=0`
4	`SYSCALL_READ`	`read(int fd, void *buf, uint32_t len)`	`eax=SYSCALL_READ, ebx=(uint32_t)fd, ecx=(uint32_t)buf, edx=len`
5	`SYSCALL_CLOSE`	`close(int fd)`	`eax=SYSCALL_CLOSE, ebx=(uint32_t)fd, ecx=0, edx=0`
6	`SYSCALL_EXEC`	`exec(const char *path, const char *args, uint32_t args_len)`	`eax=SYSCALL_EXEC, ebx=(uint32_t)path, ecx=(uint32_t)args, edx=args_len`
7	`SYSCALL_GETARGS`	`getargs(char *buf, uint32_t len)`	`eax=SYSCALL_GETARGS, ebx=(uint32_t)buf, ecx=len, edx=0`
8	`SYSCALL_STAT`	``	`eax=SYSCALL_STAT, ebx=?, ecx=?, edx=?`
9	`SYSCALL_SEEK`	`lseek(int fd, int offset, int whence)`	`eax=SYSCALL_SEEK, ebx=(uint32_t)fd, ecx=(uint32_t)offset, edx=(uint32_t)whence`
10	`SYSCALL_LISTDIR`	`listdir(const char *path, void *entries, uint32_t max_entries)`	`eax=SYSCALL_LISTDIR, ebx=(uint32_t)path, ecx=(uint32_t)entries, edx=max_entries`
11	`SYSCALL_MKDIR`	`mkdir(const char *path)`	`eax=SYSCALL_MKDIR, ebx=(uint32_t)path, ecx=0, edx=0`
12	`SYSCALL_RM`	`rm(const char *path)`	`eax=SYSCALL_RM, ebx=(uint32_t)path, ecx=0, edx=0`
13	`SYSCALL_TOUCH`	`touch(const char *path)`	`eax=SYSCALL_TOUCH, ebx=(uint32_t)path, ecx=0, edx=0`
14	`SYSCALL_GETCWD`	`getcwd(char *buf, uint32_t len)`	`eax=SYSCALL_GETCWD, ebx=(uint32_t)buf, ecx=len, edx=0`
15	`SYSCALL_SETCWD`	`setcwd(const char *path)`	`eax=SYSCALL_SETCWD, ebx=(uint32_t)path, ecx=0, edx=0`
16	`SYSCALL_CLEAR`	`clear(void)`	`eax=SYSCALL_CLEAR, ebx=0, ecx=0, edx=0`
17	`SYSCALL_SETCOLOR`	`setcolor(uint32_t fg, uint32_t bg)`	`eax=SYSCALL_SETCOLOR, ebx=fg, ecx=bg, edx=0`
18	`SYSCALL_WRITEFILE`	`writefile(const char *path, const void *buf, uint32_t len)`	`eax=SYSCALL_WRITEFILE, ebx=(uint32_t)path, ecx=(uint32_t)buf, edx=len`
19	`SYSCALL_HISTORY_COUNT`	`history_count(void)`	`eax=SYSCALL_HISTORY_COUNT, ebx=0, ecx=0, edx=0`
20	`SYSCALL_HISTORY_GET`	`history_get(uint32_t index, char *buf, uint32_t len)`	`eax=SYSCALL_HISTORY_GET, ebx=index, ecx=(uint32_t)buf, edx=len`
21	`SYSCALL_GET_TICKS`	`get_ticks(void)`	`eax=SYSCALL_GET_TICKS, ebx=0, ecx=0, edx=0`
22	`SYSCALL_GET_COMMAND_COUNT`	`get_command_count(void)`	`eax=SYSCALL_GET_COMMAND_COUNT, ebx=0, ecx=0, edx=0`

#	Syscall macro	Userspace wrapper	Register mapping (from wrapper)
23	SYSCALL_GETCHAR	getchar(void)	eax=SYSCALL_GETCHAR, ebx=0, ecx=0, edx=0
24	SYSCALL_SLEEP_MS	sleep_ms(uint32_t ms)	eax=SYSCALL_SLEEP_MS, ebx=ms, ecx=0, edx=0
25	SYSCALL_ALIAS_SET	alias_set(const char *name, const char *cmd)	eax=SYSCALL_ALIAS_SET, ebx=(uint32_t)name, ecx=(uint32_t)cmd, edx=0
26	SYSCALL_ALIAS_REMOVE	alias_remove(const char *name)	eax=SYSCALL_ALIAS_REMOVE, ebx=(uint32_t)name, ecx=0, edx=0
27	SYSCALL_ALIAS_COUNT	alias_count(void)	eax=SYSCALL_ALIAS_COUNT, ebx=0, ecx=0, edx=0
28	SYSCALL_ALIAS_GET	alias_get(uint32_t index, char *name, char *cmd)	eax=SYSCALL_ALIAS_GET, ebx=index, ecx=(uint32_t)name, edx=(uint32_t)cmd
29	SYSCALL_TIMER_START	timer_start(void)	eax=SYSCALL_TIMER_START, ebx=0, ecx=0, edx=0
30	SYSCALL_TIMER_STOP	timer_stop(void)	eax=SYSCALL_TIMER_STOP, ebx=0, ecx=0, edx=0
31	SYSCALL_TIMER_STATUS	timer_status(void)	eax=SYSCALL_TIMER_STATUS, ebx=0, ecx=0, edx=0
32	SYSCALL_BEEP	beep(uint32_t frequency_hz, uint32_t duration_ms)	eax=SYSCALL_BEEP, ebx=frequency_hz, ecx=duration_ms, edx=0
33	SYSCALL_HALT	halt(void)	eax=SYSCALL_HALT, ebx=0, ecx=0, edx=0
34	SYSCALL_GFX_DEMO	gfx_demo(void)	eax=SYSCALL_GFX_DEMO, ebx=0, ecx=0, edx=0
35	SYSCALL_GFX_ANIM	gfx_anim(void)	eax=SYSCALL_GFX_ANIM, ebx=0, ecx=0, edx=0
36	SYSCALL_GFX_PAINT	gfx_paint(const char *path)	eax=SYSCALL_GFX_PAINT, ebx=(uint32_t)path, ecx=0, edx=0
37	SYSCALL_GUI_DESKTOP	gui_desktop(void)	eax=SYSCALL_GUI_DESKTOP, ebx=0, ecx=0, edx=0
38	SYSCALL_GUI	gui_run(void)	eax=SYSCALL_GUI, ebx=0, ecx=0, edx=0
39	SYSCALL_GUI_PAINT	gui_paint(const char *path)	eax=SYSCALL_GUI_PAINT, ebx=(uint32_t)path, ecx=0, edx=0
40	SYSCALL_GUI_CALC	gui_calc(void)	eax=SYSCALL_GUI_CALC, ebx=0, ecx=0, edx=0
41	SYSCALL_GUI_FILEMGR	gui_filemgr(void)	eax=SYSCALL_GUI_FILEMGR, ebx=0, ecx=0, edx=0
42	SYSCALL_GFX_SET_MODE	``	eax=SYSCALL_GFX_SET_MODE, ebx=?, ecx=?, edx=?
43	SYSCALL_GFX_GET_MODE	``	eax=SYSCALL_GFX_GET_MODE, ebx=?, ecx=?, edx=?
44	SYSCALL_GFX_GET_WIDTH	``	eax=SYSCALL_GFX_GET_WIDTH, ebx=?, ecx=?, edx=?
45	SYSCALL_GFX_GET_HEIGHT	``	eax=SYSCALL_GFX_GET_HEIGHT, ebx=?, ecx=?, edx=?
46	SYSCALL_GFX_CLEAR	``	eax=SYSCALL_GFX_CLEAR, ebx=?, ecx=?, edx=?
47	SYSCALL_GFX_PUTPIXEL	``	eax=SYSCALL_GFX_PUTPIXEL, ebx=?, ecx=?, edx=?
48	SYSCALL_GFX_DRAW_RECT	``	eax=SYSCALL_GFX_DRAW_RECT, ebx=?, ecx=?, edx=?
49	SYSCALL_GFX_FILL_RECT	``	eax=SYSCALL_GFX_FILL_RECT, ebx=?, ecx=?, edx=?
50	SYSCALL_GFX_DRAW_LINE	``	eax=SYSCALL_GFX_DRAW_LINE, ebx=?, ecx=?, edx=?
51	SYSCALL_GFX_DRAW_CHAR	``	eax=SYSCALL_GFX_DRAW_CHAR, ebx=?, ecx=?, edx=?
52	SYSCALL_GFX_PRINT	``	eax=SYSCALL_GFX_PRINT, ebx=?, ecx=?, edx=?

#	Syscall macro	Userspace wrapper	Register mapping (from wrapper)
53	`SYSCALL_GFX_FLIP`	``	`eax=SYSCALL_GFX_FLIP, ebx=?, ecx=?, edx=?`
54	`SYSCALL_GFX_DOUBLEBUFFER_ENABLE`	``	`eax=SYSCALL_GFX_DOUBLEBUFFER_ENABLE, ebx=?, ecx=?, edx=?`
55	`SYSCALL_GFX_DOUBLEBUFFER_DISABLE`	``	`eax=SYSCALL_GFX_DOUBLEBUFFER_DISABLE, ebx=?, ecx=?, edx=?`
56	`SYSCALL_MOUSE_GET_STATE`	``	`eax=SYSCALL_MOUSE_GET_STATE, ebx=?, ecx=?, edx=?`
57	`SYSCALL_KEYBOARD_HAS_INPUT`	`keyboard_has_input(void)`	`eax=SYSCALL_KEYBOARD_HAS_INPUT, ebx=0, ecx=0, edx=0`
58	`SYSCALL_RENAME`	`rename(const char *old_path, const char *new_name)`	`eax=SYSCALL_RENAME, ebx=(uint32_t)old_path, ecx=(uint32_t)new_name, edx=0`
59	`SYSCALL_SPAWN`	`spawn(const char *path, const char *args, uint32_t args_len)`	`eax=SYSCALL_SPAWN, ebx=(uint32_t)path, ecx=(uint32_t)args, edx=args_len`
60	`SYSCALL_WAIT`	`wait(int *status)`	`eax=SYSCALL_WAIT, ebx=(uint32_t)-1, ecx=(uint32_t)status, edx=0`
61	`SYSCALL_FORK`	`fork(void)`	`eax=SYSCALL_FORK, ebx=0, ecx=0, edx=0`
62	`SYSCALL_GFX_BLIT`	``	`eax=SYSCALL_GFX_BLIT, ebx=?, ecx=?, edx=?`
63	`SYSCALL_KEY_REPEAT`	`keyboard_set_repeat(uint8_t delay, uint8_t rate)`	`eax=SYSCALL_KEY_REPEAT, ebx=(uint32_t)delay, ecx=(uint32_t)rate, edx=0`
64	`SYSCALL_SPEAKER_START`	`speaker_start(uint32_t frequency_hz)`	`eax=SYSCALL_SPEAKER_START, ebx=frequency_hz, ecx=0, edx=0`
65	`SYSCALL_SPEAKER_STOP`	`speaker_stop(void)`	`eax=SYSCALL_SPEAKER_STOP, ebx=0, ecx=0, edx=0`
66	`SYSCALL_FS_FREE_BLOCKS`	`fs_get_free_blocks(void)`	`eax=SYSCALL_FS_FREE_BLOCKS, ebx=0, ecx=0, edx=0`
67	`SYSCALL_HEAP_STATS`	`heap_get_stats(user_heap_stats_t *stats)`	`eax=SYSCALL_HEAP_STATS, ebx=(uint32_t)stats, ecx=sizeof(*stats), edx=0`
68	`SYSCALL_PROCESS_COUNT`	`process_count(void)`	`eax=SYSCALL_PROCESS_COUNT, ebx=0, ecx=0, edx=0`
69	`SYSCALL_PROCESS_LIST`	`process_list(user_process_info_t *out, uint32_t max_entries)`	`eax=SYSCALL_PROCESS_LIST, ebx=(uint32_t)out, ecx=max_entries, edx=0`
70	`SYSCALL_INSTALL_EMBEDDED`	`install_embedded(const char *path)`	`eax=SYSCALL_INSTALL_EMBEDDED, ebx=(uint32_t)path, ecx=0, edx=0`
71	`SYSCALL_BRK`	`brk(void *addr)`	`eax=SYSCALL_BRK, ebx=(uint32_t)addr, ecx=0, edx=0`
72	`SYSCALL_PIPE`	`pipe(int fds[2])`	`eax=SYSCALL_PIPE, ebx=(uint32_t)fds, ecx=0, edx=0`
73	`SYSCALL_DUP2`	`dup2(int oldfd, int newfd)`	`eax=SYSCALL_DUP2, ebx=(uint32_t)oldfd, ecx=(uint32_t)newfd, edx=0`
74	`SYSCALL_KILL`	`kill(int pid, int sig)`	`eax=SYSCALL_KILL, ebx=(uint32_t)pid, ecx=(uint32_t)sig, edx=0`
75	`SYSCALL_AUDIO_WRITE`	`audio_write(const void *buf, uint32_t bytes)`	`eax=SYSCALL_AUDIO_WRITE, ebx=(uint32_t)buf, ecx=bytes, edx=0`
76	`SYSCALL_AUDIO_SET_VOLUME`	`audio_set_volume(uint8_t master, uint8_t pcm)`	`eax=SYSCALL_AUDIO_SET_VOLUME, ebx=master, ecx=pcm, edx=0`
77	`SYSCALL_AUDIO_GET_VOLUME`	`audio_get_volume(uint8_t *master, uint8_t *pcm)`	`eax=SYSCALL_AUDIO_GET_VOLUME, ebx=0, ecx=0, edx=0`

#	Syscall macro	Userspace wrapper	Register mapping (from wrapper)
78	`SYSCALL_AUDIO_STATUS`	`audio_is_ready(void)`	`eax=SYSCALL_AUDIO_STATUS, ebx=0, ecx=0, edx=0`